Preparing your laptop for Docker

Docker lets you run software in **containers**: self-contained environments that include an application and the software it needs. We recommend one of the following two supported ways to prepare your laptop to use Docker:

1. **Multipass + Ubuntu + Docker Engine:** install Multipass, create an Ubuntu virtual machine (VM), and install Docker inside it. This gives you a separate Linux environment.
2. **Docker Desktop:** install Docker Desktop directly on your laptop. It manages the Linux environment needed to run Linux containers.

For most students who want the fewest setup steps, **Docker Desktop is usually the easiest choice**. Multipass is useful if you want a separate Ubuntu environment while keeping the VM setup as similar as possible across Windows, macOS and Linux.

Before you start

1. **Check your operating system and processor.**
   - Windows: Settings → System → About.
   - macOS: Apple menu → About This Mac. Check whether the chip is Apple (M-series) or Intel.
   - Linux: open a terminal and run `uname -m`. `x86_64` means Intel/AMD 64-bit; `aarch64` or `arm64` means ARM 64-bit.
2. **Update your operating system** and connect your laptop to power.
3. **Check available resources.** Ideally have at least 8 GB RAM on the laptop and 20–30 GB of free disk space. A VM needs additional disk space and memory.
4. **Check permissions.** Installing virtualization software or enabling virtualization features may require an administrator password.
5. Download software only from the official links in this guide.

Option 1 — Multipass, Ubuntu and Docker Engine

Multipass is a tool from Canonical (the company behind Ubuntu) that creates and manages Ubuntu virtual machines. Unlike traditional VM software, it downloads and installs Ubuntu automatically: you do not need to download an ISO file or use the Ubuntu graphical installer.

The commands used to create and manage the VM are almost identical on Windows, macOS and Linux. Multipass uses the host computer's native virtualization technology behind the scenes. The Ubuntu VM is normally a **server installation without a graphical desktop**; you work with it through a terminal. You can still use your normal host applications (such as a code editor or file manager) and share a project folder with the VM (see Section 5).

> **GUI or terminal?** Multipass includes a small graphical/tray application, but it is not a full virtual-machine manager with all settings available in menus. The GUI can open a shell and create a default `primary` VM. For a VM with the recommended memory, CPU and disk settings, the most reliable method is to copy one short command into a terminal. Both approaches are described below.

1. Install Multipass

Start from the official Multipass installation page. It detects the available operating-system instructions. Check the current requirements there before installing.

Windows 10/11

1. Make sure Windows is up to date. Multipass requires the **Virtual Machine Platform** Windows feature.
2. If it is not already enabled, open **Turn Windows features on or off** from the Start menu, select **Virtual Machine Platform**, click OK, and restart Windows if requested. Follow Microsoft's Virtual Machine Platform instructions.
3. Download the Windows installer from the official Multipass installation page.
4. Run the downloaded `.msi` installer. Approve the administrator prompt and follow the installation wizard.
5. Open **PowerShell** or **Windows Terminal** and check the installation:

```
multipass version
```

macOS (Apple silicon or Intel)

1. Check that your Mac is running **macOS 14 Sonoma or later** (the current Multipass requirement).
2. Download the macOS installer (`.pkg`) from the official Multipass installation page.
3. Open the `.pkg` file and follow the installer. An administrator password may be required.
4. Open **Terminal** (Applications → Utilities → Terminal) and check the installation:

```
multipass version
```

Multipass supports both Intel and Apple-silicon Macs. It selects the appropriate virtualization backend and Ubuntu image; students should not need to choose an AMD64 or ARM64 ISO manually.

Linux (Ubuntu and other distributions)

Multipass is distributed as a Snap package. On Ubuntu, `snapd` is normally already installed.

1. Open a terminal.
2. Install Multipass:

```
sudo snap install multipass
```

3. Check the installation:

```
multipass version
```

On other Linux distributions, first install and configure `snapd` by following the Snap installation instructions, then install Multipass as above. If your university laptop does not allow Snap packages, ask university IT for help.

2. Create and start the Ubuntu VM

You can use either the Multipass graphical app or the command line. **For this course, we recommend the command-line method**, because it creates a VM with enough resources using one consistent command on all operating systems.

Method A — Graphical interface (quickest, default settings)

1. Open **Multipass** from the Applications menu (Linux), Start menu (Windows), or Applications folder (macOS).
2. Find the Multipass icon in the system tray/menu bar. On Windows it may be in the hidden-icons area near the clock.
3. Select **Open Shell**.
4. Multipass creates and starts a VM named `primary`, installs the latest Ubuntu LTS image, and opens a terminal inside it.

Important: the default `primary` VM is intentionally small (typically 1 CPU, 1 GB RAM and 5 GB disk). This is useful for a quick test, but it may be too small for data-science exercises. If you use this GUI shortcut, ask your instructor before relying on it for the full course. To create the recommended VM, use Method B below instead.

Method B — Terminal (recommended course setup)

Open a terminal on your host computer:

- Windows: **PowerShell** or **Windows Terminal**.
- macOS: **Terminal** (Applications → Utilities → Terminal).
- Linux: your usual terminal application.

Copy and run this command:

```
multipass launch lts --name tids-ubuntu --cpus 2 --memory 4G --disk 30G
```

This creates and starts an Ubuntu LTS VM named `tids-ubuntu` with 2 CPU cores, 4 GB RAM and a 30 GB disk. The first launch downloads the Ubuntu image and can take a few minutes. If your laptop has 16 GB RAM or more, you may increase the VM memory to `6G` or `8G`; do not allocate all your laptop's memory or CPU cores to the VM.

To check that the VM is running:

```
multipass list
multipass info tids-ubuntu
```

3. Open the Ubuntu terminal

Graphical method

1. Open the Multipass tray/menu-bar icon.
2. Choose the running `tids-ubuntu` instance, if shown, and select **Open Shell**.
3. If the GUI does not show the instance or shell option, use the terminal method below.

Terminal method

On the host computer, run:

```
multipass shell tids-ubuntu
```

You are now typing commands **inside Ubuntu**, not in Windows/macOS/Linux on your laptop. The prompt will usually look similar to `ubuntu@tids-ubuntu:~$`.

Update Ubuntu:

```
sudo apt update
sudo apt upgrade -y
```

Enter the Ubuntu password only if prompted. In a standard Multipass Ubuntu instance, the default `ubuntu` account is configured for administrative commands through `sudo`; you may not be asked for a password.

4. Install Docker Engine in Ubuntu

Use Docker's official Ubuntu instructions:
Install Docker Engine on Ubuntu — Docker documentation.

The recommended method is to add Docker's official `apt` repository and install the Docker Engine packages. Follow the current instructions on that page in order. In particular:

1. Remove conflicting unofficial/older Docker packages if the instructions say they are present.
2. Set up Docker's official `apt` repository and signing key using the commands shown on the Docker page.
3. Install the packages listed under **Install Docker Engine**. These include the Docker Engine, command-line client, container runtime, Buildx plugin and Compose plugin.
4. Start/enable the Docker service if the installation instructions indicate it is not already running.

Do not copy installation commands from random blogs. Docker's official page is kept up to date and may change as Ubuntu releases change.

5. Share a project folder between your laptop and Ubuntu

A shared folder is useful because you can edit notebooks, Python files and course materials using applications installed on your laptop, while running Docker commands inside Ubuntu. Changes made on either side appear in the same shared directory.

Create a folder on your laptop first, for example:

- Windows: `C:\Users\<your-user>\tids-course`
- macOS/Linux: `~/tids-course`

Replace `<your-user>` with your Windows account folder name. Put your course files in this folder.

Share the folder using the terminal (recommended)

The command format is the same on Windows, macOS and Linux:

```
multipass mount "<HOST_FOLDER_PATH>" tids-ubuntu:/home/ubuntu/course
```

Replace `<HOST_FOLDER_PATH>` with the full path to the folder you created on your laptop. Examples:

Windows PowerShell:

```
multipass mount "C:\Users\Alice\tids-course" tids-ubuntu:/home/ubuntu/course
```

macOS:

```
multipass mount "$HOME/tids-course" tids-ubuntu:/home/ubuntu/course
```

Linux:

```
multipass mount "$HOME/tids-course" tids-ubuntu:/home/ubuntu/course
```

Then, inside the Ubuntu VM, access the shared folder:

```
cd /home/ubuntu/course
ls
```

Files you create or edit in this directory from the host or the VM are shared. The mount remains associated with the instance and is normally restored when the instance starts again. To remove the shared mount:

```
multipass unmount tids-ubuntu:/home/ubuntu/course
```

Windows-specific note: for security reasons, Multipass mounts are disabled by default on Windows. If the mount command reports that mounts are disabled, open PowerShell as Administrator and run:

```
multipass set local.privileged-mounts= true
```

If your installed version rejects the space before `true`, use:

```
multipass set local.privileged-mounts=true
```

Then retry the `multipass mount` command. Enabling privileged mounts gives the VM access to the selected host folder, so share only a dedicated course folder, not your whole user profile or system drive. See the Multipass platform notes and mount security notes.

Share the folder using the GUI

Multipass's graphical client does not currently provide a complete folder-sharing wizard. The easiest GUI-first workflow is:

1. Create the `tids-course` folder using File Explorer (Windows), Finder (macOS), or your Linux file manager.
2. Open the host's terminal application.
3. Run the single `multipass mount` command shown above, using the path for your operating system.
4. Use your normal graphical editor and file manager on the host to work with the folder. In Ubuntu, access the same files at `/home/ubuntu/course`.

This approach keeps file editing graphical and requires only one terminal command to connect the folder. For more details, see Canonical's Share data with an instance guide.

6. Check that Docker works

Inside the Ubuntu VM, run:

```
sudo docker run hello-world
```

Docker should download a small test image and print a message confirming that the installation works.

Check the installed versions:

```
sudo docker --version
sudo docker compose version
```

For this guide, using `sudo` before Docker commands is the simplest approach and avoids changing user permissions. You may see tutorials recommending adding your account to the `docker` group to avoid typing `sudo`; be aware that membership in that group grants very powerful, effectively root-level access. Do not do this on a shared or managed machine unless your instructor or IT administrator recommends it.

7. Everyday use: start, stop and reopen the VM

- **Start the VM:** open Multipass from the application menu/tray, or run `multipass start tids-ubuntu`.
- **Open Ubuntu:** choose **Open Shell** in the Multipass tray app, or run `multipass shell tids-ubuntu`.
- **Stop the VM:** use the Multipass app if the instance controls are available, or run `multipass stop tids-ubuntu`.
- **See VM status:** run `multipass list`.
- **Remove the VM completely:** run `multipass delete --purge tids-ubuntu`. This permanently deletes the VM and its files. Copy any work you need to keep to your shared host folder first.

Run the course's Docker commands inside the Ubuntu shell. Keep notebooks and other course files in the shared `/home/ubuntu/course` directory so that they are easy to access from your laptop.

Option 2 — Install Docker Desktop directly

Docker Desktop provides the Docker command-line tools and a graphical dashboard. On Windows and macOS it manages the lightweight Linux environment required to run Linux containers. On Linux it also runs Docker Desktop in its own VM.

Start at the official Docker Desktop page, then follow the matching operating-system instructions below.

A. Windows 10/11

Docker Desktop on Windows normally uses **WSL 2** (Windows Subsystem for Linux), which provides a Linux environment without requiring you to create and manage a Multipass VM.

1. Check that your Windows version meets the current Docker Desktop for Windows requirements. Hardware virtualization must be enabled.
2. Open **PowerShell as Administrator**:
   - Right-click Start.
   - Select **Terminal (Admin)** or **Windows PowerShell (Admin)**.
3. Install or update WSL by running:

```
wsl --install
wsl --update
```

Restart Windows if requested. The command normally installs Ubuntu as the default Linux distribution.

4. Check that WSL 2 is available:

```
wsl --version
wsl --status
```

If the commands indicate that WSL needs updating, follow Microsoft's WSL installation and update guide.

5. Download Docker Desktop using the official Windows installation page.
6. Run `Docker Desktop Installer.exe`. When asked which backend to use, choose **WSL 2** (the usual/recommended choice). Complete the installation and restart if requested.
7. Open Docker Desktop from the Start menu. Accept the terms if prompted and wait until Docker reports that it is running.
8. In Docker Desktop, open **Settings** → **Resources** → **WSL Integration** and make sure integration is enabled for your Ubuntu distribution. Select **Apply** if you change anything.
9. Open Ubuntu from the Start menu (or Windows Terminal) and test:

```
docker run hello-world
docker --version
docker compose version
```

With Docker Desktop running and WSL integration enabled, you normally do **not** need to install Docker Engine separately inside Ubuntu.

Official references: Docker Desktop on Windows, Docker's WSL 2 integration guide, and Microsoft's WSL guide.

B. macOS (Apple silicon or Intel)

1. Check your Mac's processor: Apple menu → **About This Mac**.
2. Open Docker's official Install Docker Desktop on Mac page.
3. Download the correct installer:
   - **Apple chip:** choose **Mac with Apple chip**.
   - **Intel chip:** choose **Mac with Intel chip**.
4. Open the downloaded `.dmg` file and drag Docker to the **Applications** folder, following the installer instructions.
5. Open Docker from Applications. Approve any macOS permission prompts and enter your Mac password if requested. Accept the Docker terms if prompted.
6. Wait for Docker Desktop to finish starting. Its menu-bar icon should indicate that Docker is running.
7. Open **Terminal** (Applications → Utilities → Terminal) and test:

```
docker run hello-world
docker --version
docker compose version
```

Do not install the Intel version on an Apple-silicon Mac, or vice versa. Docker Desktop for Mac uses a Linux environment internally; you do not need to install a separate Ubuntu VM.

C. Linux desktop (for example, Ubuntu)

On Linux, Docker Desktop is available for selected distributions and architectures. It has more prerequisites than the Windows and Mac installers. If your Linux distribution is not supported, or you prefer a simpler Linux-native setup, use **Option 1** (Multipass + Ubuntu) or ask your instructor whether Docker Engine is appropriate for your distribution.

1. Check the supported distributions and requirements on Docker Desktop for Linux. Check the page for currently supported release versions.
2. Confirm that your machine supports hardware virtualization and KVM. Docker Desktop for Linux requires KVM, QEMU 5.2 or later, systemd, a supported desktop environment, and at least 4 GB RAM. Follow Docker's KVM setup instructions if needed.
3. Open the matching official installation page:
   - Ubuntu installation
   - Other supported Linux distributions
4. Follow Docker's instructions to configure its package repository, download the correct `.deb` (Ubuntu/Debian) or `.rpm` (Fedora/RHEL) package, and install it. Use the package for your distribution and CPU architecture.
5. Launch **Docker Desktop** from your applications menu and accept the terms if prompted. Wait until it reports that it is running.
6. Open a terminal and test:

```
docker run hello-world
docker --version
docker compose version
```

Important Linux note: Docker Desktop for Linux runs in its own VM and uses a Docker context named `desktop-linux`. It is distinct from a separately installed Docker Engine. If you already have Docker Engine installed, be aware that the two environments may contain different images and containers. See Docker's explanation of Docker Desktop on Linux.

Final check and troubleshooting

Regardless of which option you chose, these commands should work in the terminal where you intend to do your course work:

```
docker --version
docker compose version
docker run hello-world
```

If `hello-world` completes successfully, Docker can download an image and run a container.

Common issues:

- **Docker says it cannot connect to the daemon / engine:** start Docker Desktop, or (for Option 1) check that the Multipass VM is running and the Docker service is active.
- **Multipass cannot launch a VM:** check the platform prerequisites on the Multipass installation page. On Windows, confirm that Virtual Machine Platform is enabled. On macOS, check the macOS version requirement. On Linux, confirm that Snap is installed.

- **Windows folder sharing fails:** Multipass mounts are disabled by default on Windows. Enable `local.privileged-mounts` as described in Section 5, then retry.
- **Windows says WSL needs an update:** run `wsl --update` in an Administrator PowerShell window, then restart if requested.
- **VM is very slow:** close other demanding applications, check that the VM has enough memory and CPU, and make sure the laptop is plugged in.
- **Not enough disk space:** Docker images and containers can use several gigabytes. Free disk space before starting the course.

Useful official links

- Docker documentation
- Docker Desktop overview and downloads
- Docker Engine on Ubuntu
- Multipass installation
- Multipass documentation
- Multipass: share data with an instance
- Ubuntu documentation
- Microsoft WSL installation